POLY PMNA · DIPLOMA CURRICULUM REVISION 2026

Course 1131 | Semester I | Practicum | 4.5 Credits | 90 Contact Hours

Problem Solving and Python Programming

A full-screen bilingual handbook for designing reliable algorithms, drawing correct flowcharts and building readable Python programs through theory, demonstrations and laboratory practice.

Teaching scheme	**3:0:3:0**
Theory / Practical	**45 h / 45 h**
Self-learning	**6 h/week**
CIE / ESE	**40 / 100**
Total assessment	**140**
Primary CO level	**Apply**

AUTHORITY AND VERIFICATION

Official Source Declaration and Verified Inconsistencies

This handbook uses only the supplied SITTTR Kerala **Diploma Curriculum Revision 2026** PDF for Course **1131 — Problem Solving and Python Programming**. No Revision 2021 course content or another subject's curriculum has been merged.

Incomplete detailed-syllabus extract: The supplied PDF contains four pages and ends after the Module III list practical. The major-topics table on pages 2–3 confirms all four modules and their theory/practical hours, but the detailed table does not show the remaining Module III rows or any detailed Module IV rows. Those parts of this handbook are therefore developed only from the official major-topics and laboratory-experiments rows.

Teaching-scheme label mismatch: Page 1 labels the fourth teaching field as *R* in "L:T:P:R"; page 2 labels it *S* and defines S as Project. Both values are 0. The website displays the unaltered numeric scheme **3:0:3:0** and records the label mismatch here.

Detailed-table heading mismatch: Page 3 titles the delivery column "Mode of delivery (L/T)", while the rows use both **L** and **P**. This handbook uses L and P exactly as shown in the rows.

Taxonomy wording mismatch: The page-4 match-case learning outcome starts with "Implement", while its listed taxonomy level is "Understand". Both are reproduced in the coverage information; no silent correction is made.

Assessment presentation is consistent: Page 1 reports CIE 40 and ESE 100. Page 2 separates Theory 20+60=80 and Practical 20+40=60, giving the same overall 140 marks.

Not specified in the supplied official syllabus PDF: prerequisite course code, prescribed textbooks, reference books, websites, detailed equipment specification, exact software version and an official model-question-paper pattern.

COURSE DASHBOARD

Official Course Profile

90 h

Total contact hours

45 h

Theory (3 L/week)

45 h

Practical (3 P/week)

4.5

Credits

Programmes

Artificial Intelligence; Artificial Intelligence & Machine Learning; Cloud Computing and Big Data; Cyber Forensics and Information Security; Computer Science and Technology; Computer Science & Engineering; Computer Engineering; Computer Science and Engineering (Artificial Intelligence & Machine Learning); Information Technology; Automation and Robotics; Robotic Process Automation.

Prerequisite

Topic: Basic understanding of computer terminologies.

Course title shown: IT lab taught in secondary level school.

Semester: Secondary Education.

Course code: Not specified in the supplied official syllabus PDF.

Official teaching and assessment scheme

L	T	P	S/R	SS/week	Total hours	Credits	Theory CIE	Theory ESE	Practical CIE	Practical ESE	Total
3	0	3	0	6	90	4.5	20	60	20	40	140

How to use this handbook

1. Understand

Read the concept and definitions.

2. Visualise

Study the flowchart, data-flow or animation.

3. Implement

Type and run the code rather than copying output.

4. Verify

Use test cases, answer questions and complete the record.

OFFICIAL INTENT

Course Description, Objectives and Outcomes

Course description

This foundational Python course emphasises logical problem-solving and algorithmic thinking. Students master control structures, functions and data structures while gaining hands-on experience in debugging and efficient code. Practical application through laboratory experiments and self-learning enables students to develop simple programs and apply concepts to real-life problems.

Official course objectives

1. To equip students with the ability to solve complex problems by designing structured algorithms and visual flowcharts before implementation.
2. To provide a comprehensive understanding of Python syntax, including variables, data types, operators and control structures such as selection and iteration.
3. To familiarise students with manipulation of strings, lists, sets, tuples and dictionaries for efficient data management.
4. To introduce modularity through built-in and user-defined functions, focusing on code reusability and scope management.
5. To foster practical programming skills through laboratory experiments and real-world problem-solving.

Official Course Outcomes

CO1: Acquire foundational logical reasoning skills by designing flowcharts and algorithms and develop Python programs that effectively incorporate variables, fundamental data types and standard input/output operations.

CO2: Optimise program execution through strategic use of control structures and loop-control statements.

CO3: Implement effective data-management strategies using strings, lists, tuples, sets and dictionaries by applying operations and built-in methods to solve real-world programming problems.

CO4: Design and develop modular programs using functions, modules and packages with proper scope management and argument-passing techniques.

Official Bloom's level for CO1–CO4: Apply.

Official CO-PO articulation matrix (1 Low, 2 Medium, 3 High, – No correlation)											
CO	**PO1**	**PO2**	**PO3**	**PO4**	**PO5**	**PO6**	**PO7**	**PO8**	**PO9**	**PO10**	**PO11**
CO1	3	3	2	2	2	–	–	–	–	–	3
CO2	3	3	3	2	2	–	–	–	–	–	3
CO3	3	3	3	3	3	–	–	–	–	–	3
CO4	3	2	3	3	3	–	–	–	–	–	3
Correlation level	3	2.75	2.75	2.5	2.5	0	0	0	0	0	3

TRACEABILITY

Curriculum Coverage Map

Module-to-handbook traceability					
Module	**Official topics**	**Hours L+P**	**CO**	**Detailed taxonomy shown**	**Handbook chapter**
I	Problem solving, methods, development cycle, analysis, algorithms, flowcharts, Python introduction, versions, installation, IDE/interpreter, variables, literals, keywords, types, conversion, operators, I/O	12+12	CO1	Understand and Apply	Module I
II	if, if-else, nested if, if-elif, match-case, iteration, for, while, range, nested loops, break, continue, pass, practical series test	12+12	CO2	Understand and Apply	Module II
III	Strings, indexing, slicing, methods, lists, tuples, sets, dictionaries and real-life applications	12+12	CO3	Apply for the visible string/list rows; remaining detailed rows not supplied	Module III
IV	Built-in and user-defined functions, return values, positional/default arguments, local/global scope, modules, packages and NumPy introduction	9+9	CO4	Not visible in the supplied detailed table	Module IV

LEARNING ROADMAP

From an Unclear Problem to a Reusable Program

1
Define problem

2
Analyse input/output

3
Design algorithm

4
Draw flowchart

5
Implement and test

6
Organise and reuse

Malayalam support: Problem □□□□□□□□ code □□□□□□□□□. □□□□□ input, output, constraints □□□□□□□□□□□□□; □□□□ Algorithm/Flowchart □□□□□□□□□□□ test data □□□□□□□□□□ verify □□□□□ □□□□□□□ Python implementation.

MODULE I

Problem Solving & Python Fundamentals

This module develops the disciplined path from problem definition to a working Python program.

12 L | 12 P | CO1 | Understand + Apply

Module learning objectives

- Distinguish algorithmic and heuristic approaches.
- Use the complete program development cycle.
- Write finite algorithms and standards-based flowcharts.
- Install and verify Python/IDE environments.
- Use variables, data types, conversion, operators and standard I/O correctly.

1. Problem definition and problem-solving methods–

A **problem** is a gap between the current state and a required result. A programming problem must state valid inputs, expected outputs, processing rules and constraints. An **algorithmic method** is deterministic and reproducible for the defined case. A **heuristic** is a practical shortcut or informed strategy used when a complete exact method is expensive or unavailable.

Aspect	Algorithmic	Heuristic
Steps	Precisely defined	Rule-of-thumb or guided search
Guarantee	Produces defined result for valid input	May produce a useful but not optimal result
Repeatability	High	Depends on strategy and choices

Aspect	Algorithmic	Heuristic
Course example	Simple-interest calculation	Choosing test cases based on likely failure points

> **Malayalam support:** Algorithmic method-□ □□□ step-□□□ □□□□□□□□□□. Heuristic □□□□□□ experience □□□□□□□□□□□□□□□ practical shortcut □□□; best answer guarantee □□□□□□□□□□□□□□□.

2. Program development cycle and problem analysis–

1. **Problem definition:** State exactly what must be solved.
2. **Analysis:** Identify inputs, outputs, constraints, formulas and exceptional cases.
3. **Design:** Prepare algorithm, pseudocode and flowchart.
4. **Coding:** Translate design into Python with readable names.
5. **Testing and debugging:** Compare actual output with expected output for normal, boundary and invalid cases.
6. **Documentation:** Record purpose, assumptions, code and tests.
7. **Maintenance:** Correct defects and adapt requirements without breaking existing behaviour.

> **Engineering habit:** A program is not “correct” because it ran once. Correctness requires evidence from planned test cases.

3. Algorithm design, properties and conventions–

A useful algorithm has **finiteness**, **definiteness**, clearly specified **input**, at least one **output** and **effectiveness**—every step can actually be performed.

Algorithm: simple interest

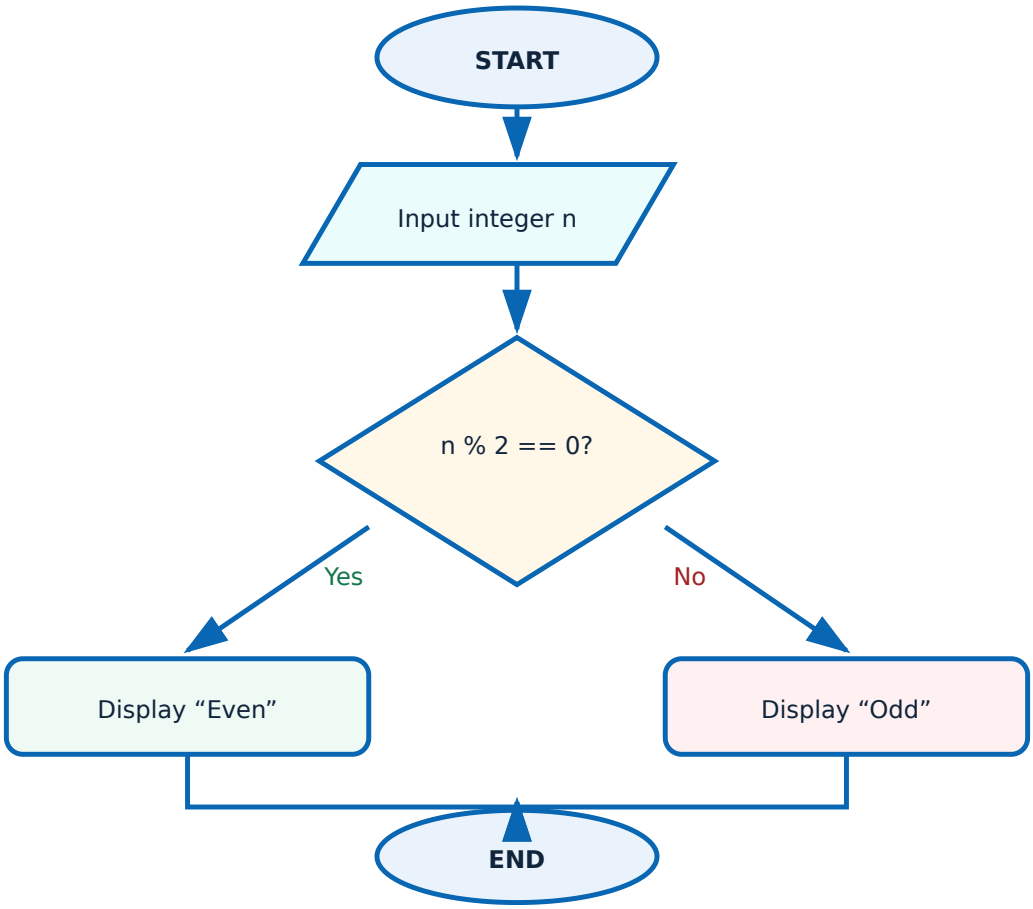
1. Start.
2. Read principal P, rate R and time T.
3. Compute SI = P × R × T / 100.
4. Display SI with a clear label.
5. End.

> **SI = (P × R × T) / 100**
> P: principal amount; R: annual percentage rate; T: time in years under the stated assumption.

Common mistake: Writing Python syntax as the algorithm. The algorithm should remain language-independent enough to verify the logic.

4. Flowchart symbols, meanings and odd/even decision–

Symbol	Shape	Purpose
Terminator	Oval	Start or End
Input/Output	Parallelogram	Read or display data
Process	Rectangle	Calculation or assignment
Decision	Diamond	Boolean test with labelled branches
Flow line	Arrow	Direction of execution



Both decision exits are labelled and both paths rejoin before End.

Malayalam support: Decision diamond-□ condition □□□□□□. Yes/No branches label □□□□□□□□ □□□□□□□□□ flowchart ambiguity □□□□□□□□□□.

5. Python introduction, features, versions and applications–

Python is a high-level, general-purpose language known for readable syntax, interactive use and a large ecosystem. Common applications include automation, data processing, web back ends, scientific computing, artificial intelligence, testing and scripting. The syllabus asks students to recognise Python versions; practical work should use an institution-approved Python 3 release because syntax and package compatibility may vary.

Readable

Indentation expresses block structure.

Portable

Source can run on different operating systems with compatible environments.

Extensible ecosystem

Standard-library modules and third-party packages support many domains.

Industrial application: A technician can automate CSV test-result checking, rename drawing exports or validate configuration files. The same logic begins with the fundamentals taught here.

6. Installation, interpreter and IDE verification–

The **Python interpreter** executes code. An **IDE** provides editing, run/debug controls and project management. IDLE is commonly bundled; PyCharm and VS Code are separate development environments.

1. Install an institution-approved Python 3 release.
2. Verify using `python --version` or `py --version`.
3. Create `hello.py` in the assigned folder.
4. Run the file from the IDE and, where instructed, from the terminal.
5. Confirm the IDE's selected interpreter matches the verified environment.

```
print("Python is ready")
```

Malayalam support: IDE code □□□□□□□□ debug □□□□□□□□□□□□□ environment □□□. Interpreter □□□ Python statements execute □□□□□□□□□□□. IDE □□□□□□□ interpreter

select □□□□□□□ package mismatch □□□□.

7. Variables, keywords, literals and built-in data types–

A variable name is bound to an object. Names may contain letters, digits and underscore but cannot start with a digit or be a Python keyword. A **literal** is a value written directly in source code.

```
student_name = "Anu"   # str literal
age = 18                 # int literal
temperature = 36.5       # float literal
is_ready = True          # bool literal
missing_value = None     # NoneType object
```

Type	Example	Typical use
int	25	Counts and whole numbers
float	25.4	Measurements and decimal calculations
bool	True	Conditions and state
str	"Motor A"	Text
NoneType	None	No value / not yet available

> Keywords such as `if`, `for`, `def` and `return` cannot be used as variable names.

8. type(), conversion and casting–

`type()` reports an object's type. Constructors such as `int()`, `float()` and `str()` perform explicit conversion when the value is valid.

```
raw = input("Quantity: ")
print(type(raw))          # str
quantity = int(raw)
print(type(quantity))     # int
price = float("12.50")
label = str(125)
```

> **Conversion is not validation by itself:** `int("12.5")` raises ValueError. Decide the accepted format and handle invalid text clearly.

9. Operators in Python–

Group	Operators	Purpose
Arithmetic	+ - * / // % **	Numeric calculations
Assignment	= += -= *= /=	Bind/update a name
Comparison	== != < <= > >=	Produce Boolean results
Logical	and or not	Combine/negate conditions
Identity	is, is not	Compare object identity
Membership	in, not in	Test collection membership
Bitwise	& | ^ ~ << >>	Operate on integer bit patterns

> Use parentheses when the intended order is important. Do not rely on memory of precedence in safety-critical or examination code.

10. Input, output, comments and f-strings–

`input()` obtains text from the user. `print()` displays values. A # comment documents intent; it should explain *why*, not repeat obvious syntax. f-strings embed expressions and control formatting.

```
principal = float(input("Principal amount: "))
rate = float(input("Rate (% per year): "))
time = float(input("Time (years): "))
interest = principal * rate * time / 100
print(f"Simple interest = {interest:.2f}")
```

> **Output quality:** “1600” is weaker than “Simple interest = 1600.00”. Labels and units are part of correct engineering communication.

Solved demonstration: Celsius to Fahrenheit

Problem: Convert 25 °C to Fahrenheit.

Given: C = 25 °C. **Required:** F.

> $F = (9/5)C + 32$

Substitution: F = (9/5 × 25) + 32 = 45 + 32.

> **Final answer:** 77 °F.

```
c = float(input("Celsius: "))
f = (9 / 5) * c + 32
print(f"{c:.1f} °C = {f:.1f} °F")
```

Interactive simple-interest check

Principal

10000

Rate %

8

Time

2

SI = 1600.00

Malayalam support: Calculator answer copy □□□□□□□□□□□□□□□□; formula substitution verify □□□□□□□□□□.

Module I summary: Define → analyse → design → code → test → document. Convert input deliberately, use clear names and display labelled output.

MODULE II

Control Structures & Looping Structure

Control structures determine which statements execute and how many times they execute.

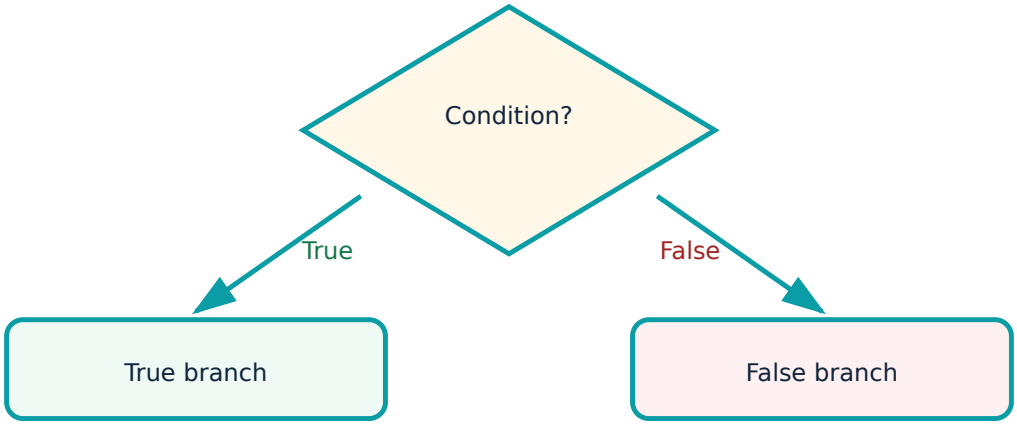
12 L | 12 P | CO2 | Understand + Apply

Module learning objectives

- Choose the correct selection structure.
- Implement for, while, range() and nested loops.
- Use break, continue and pass accurately.
- Validate fields and debug common control-flow faults.

1. Selection purpose, types and flow–

Selection evaluates a condition and chooses a path. One-way selection uses `if`; binary selection uses `if-else`; multi-way selection uses `if-elif-else`, nested decisions or `match-case`.



Binary decision: exactly one branch executes.

2. if and if-else–

```
number = int(input("Integer: "))
if number % 2 == 0:
    print("Even")
else:
    print("Odd")
```

Indentation is part of Python syntax. Statements at the same indentation belong to the same block.

> **Malayalam support:** Condition True □□□□□□□□ if block; False □□□□□□□□ else block execute □□□□□□□. Indentation □□□□□□□□□ logic □□□□□ □□□□□.

3. Nested if and if-elif–

A nested `if` is useful when the second decision is meaningful only after the first. An `if-elif-else` chain is clearer for ordered, mutually exclusive alternatives.

```
mark = float(input("Mark: "))
if not 0 <= mark <= 100:
    print("Invalid mark")
elif mark >= 90:
    print("Grade A")
elif mark >= 75:
    print("Grade B")
elif mark >= 60:
    print("Grade C")
else:
    print("Needs improvement")
```

> Order conditions from most restrictive/highest threshold to least. Starting with `mark >= 60` would capture higher grades too early.

4. match-case syntax and application–

```
day = int(input("Day number (1-7): "))
match day:
    case 1: print("Monday")
    case 2: print("Tuesday")
    case 3: print("Wednesday")
    case 4: print("Thursday")
    case 5: print("Friday")
    case 6: print("Saturday")
    case 7: print("Sunday")
    case _: print("Invalid day number")
```

`case _` is the default. The official detailed table lists taxonomy **Understand** even though its learning-outcome sentence uses the verb “Implement”; this discrepancy is retained in the source declaration.

5. Iteration purpose and for loop–

Iteration repeats a block. A `for` loop consumes items from an iterable and is suitable when the sequence or range is known.

```
number = int(input("Number: "))
for multiplier in range(1, 11):
    print(f"{number} × {multiplier} = {number * multiplier}")
```

Predict the number of iterations before running. This catches many off-by-one errors.

6. range() including negative step–

Expression	Generated values
range(5)	0, 1, 2, 3, 4
range(2, 6)	2, 3, 4, 5
range(2, 11, 3)	2, 5, 8
range(5, 0, -1)	5, 4, 3, 2, 1

The stop value is excluded. A negative step requires a start greater than stop for a non-empty descending range.

7. Nested loops and pattern printing–

```
rows = 5
for row in range(1, rows + 1):
    for column in range(row):
        print("*", end=" ")
    print()
```

The outer loop selects each row; the inner loop controls the number of symbols in that row. Total symbols for rows 1 to 5 are 1+2+3+4+5 = 15.

8. while loop–

`while` is appropriate when repetition depends on a changing condition rather than a fixed sequence.

```
count = 1
while count <= 5:
    print(count)
    count += 1
```

Infinite-loop check: Identify the control variable, initial value, condition and update. All four must be correct.

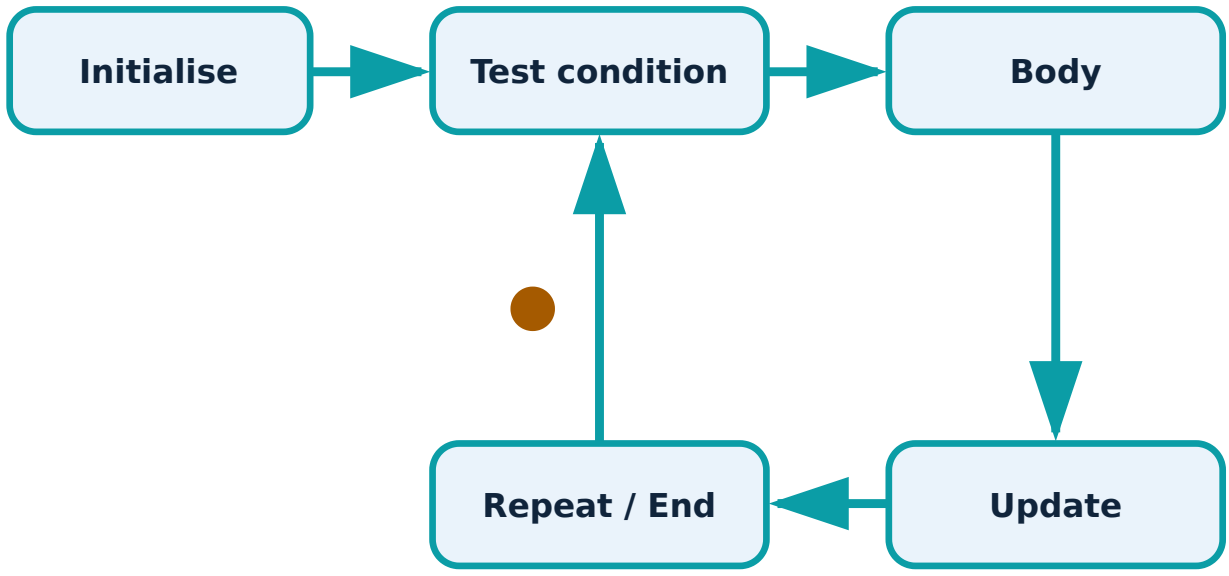
9. break, continue and pass–

Statement	Effect	Typical use
break	Exit nearest loop	Target found or stop condition reached
continue	Skip rest of current iteration	Ignore invalid/undesired item
pass	No operation	Temporary placeholder for syntactically required block

```
for n in range(1, 11):
    if n % 3 == 0:
        continue
    if n > 8:
        break
    print(n)
```

Controlled animation: one for-loop traversal

A token moves through initialise → test → body → update. Pause and inspect the static logic before coding.



Validated simple interest

```
p = float(input("Principal: "))
r = float(input("Rate: "))
t = float(input("Time: "))
if p <= 0:
    print("Principal must be positive")
elif r < 0:
    print("Rate cannot be negative")
elif t <= 0:
    print("Time must be positive")
else:
    print(f"SI = {p*r*t/100:.2f}")
```

First Series Test — practical preparation

Prepare one program each for variables/I/O, if-elif, match-case, for, while and validation. For every program include algorithm, source, three test cases and actual output.

Module II summary: Use selection for choosing paths and iteration for repetition. Treat condition order, indentation, bounds and state update as testable design decisions.

MODULE III

Strings & Data Structures — List, Tuple, Set and Dictionary

Choose a structure according to order, mutability, uniqueness and access method.

12 L | 12 P | CO3 | Apply

Module learning objectives

- Use string operations, indexing, slicing and common methods.
- Store and modify sequential data with lists.
- Use tuples for fixed ordered records.
- Apply set algebra and dictionary key-value operations.
- Select a structure for a real-life requirement.

1. Strings and string operations–

A string is an immutable sequence of Unicode characters. It supports concatenation (+), repetition (*), length (`len`) and membership (`in`).

```
course = "Python"
print(course + " Programming")
print("-" * 20)
print(len(course))
print("th" in course)
```

> **Malayalam support:** String sequence □□□□□□□□□□ immutable □□□. Original character □□□□□□□□ □□□□□□□ □□□□□□□□; operation □□□□□ string return □□□□□□□.

2. Indexing, slicing and string methods–

```
text = "  Polytechnic Python  "
clean = text.strip()
print(clean[0])
print(clean[-1])
print(clean[0:11])
print(clean[::-1])
print(clean.upper())
print(clean.lower())
print(clean.split())
print(clean.count("y"))
```

Method	Purpose	Important point
upper()/lower()	Change case in returned string	Original remains unchanged
split()	Create list of parts	Default handles runs of whitespace
strip()	Remove edge whitespace	Does not remove internal spaces
count(x)	Count non-overlapping occurrences	Case-sensitive

3. Lists: characteristics and operations–

A list is mutable, ordered and allows duplicates. It supports indexing, slicing, concatenation, repetition and membership.

```
marks = [72, 85, 61]
print(marks[0])
print(marks[-1])
print(marks[0:2])
print(marks + [90])
print(85 in marks)
```

4. List methods–

```
marks = [72, 85, 61, 72]
marks.append(90)
marks.insert(1, 78)
marks.remove(61)
last = marks.pop()
frequency = marks.count(72)
marks.sort()
print(marks, last, frequency)
```

`sort()` changes the list and returns `None`. Use `sorted(marks)` to obtain a new sorted list.

5. Tuples and their characteristics–

A tuple is ordered but immutable. It is suited to coordinates, fixed records and multiple return values.

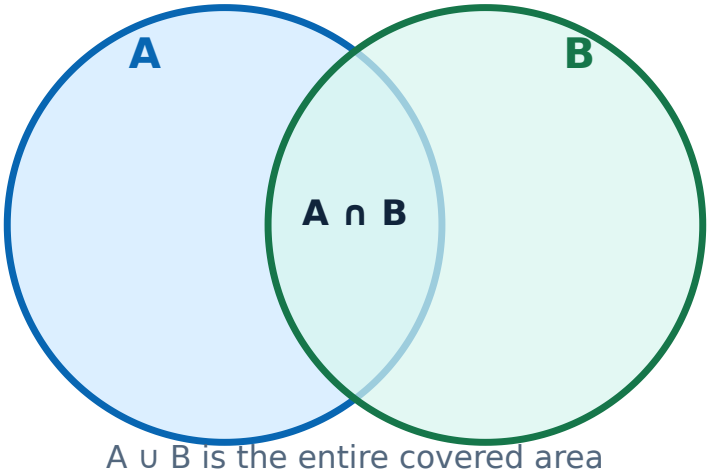
```
point = (10, 20)
x, y = point
single_item = (5,)
print(x, y, single_item)
```

The comma creates a one-item tuple: `(5,)`. Parentheses alone do not.

6. Sets and set operations–

A set stores unique, unordered elements. Index-based logic is invalid because display order is not a contract.

```
a = {1, 2, 3, 4}
b = {3, 4, 5}
print(a.union(b))
print(a.intersection(b))
print(a.difference(b))
```



A ∪ B is the entire covered area

Intersection is common membership; union contains membership from either set.

7. Dictionaries and dictionary operations–

A dictionary maps unique keys to values and is ideal for records with labelled fields.

```
student = {"name": "Anu", "mark": 84}
student["grade"] = "B"       # insert
student["mark"] = 88          # update
print(student)
print(student.keys())
print(student.values())
removed = student.pop("grade")
print(removed, student)
```

Real-life use: `{"component":"relay", "coil_voltage_v":24, "contacts":"2CO"}` preserves the meaning of each field better than relying on numeric positions.

8. Selecting the correct data structure–

Structure	Ordered	Mutable	Duplicates	Choose it when…
str	Yes	No	Yes	Data is text
list	Yes	Yes	Yes	Sequence changes
tuple	Yes	No	Yes	Fixed ordered record
set	No positional order	Yes	No	Uniqueness or set algebra matters
dict	Insertion order	Yes	Keys unique	Fields require meaningful keys

> **Malayalam support:** Ordered data change □□□□□□□□□□□□□ List; fixed record-□□ Tuple; duplicate □□□□□□□□□□ Set; labelled fields-□□ Dictionary □□□□□□□□□□□□□□□.

Demonstration: vowel count and reverse

```
text = input("Text: ")
vowels = 0
for ch in text.lower():
    if ch in "aeiou":
        vowels += 1
print("Vowels:", vowels)
print("Reverse:", text[::-1])
```

Demonstration: unique test-failure codes

```
failures = ["E01", "E02", "E01", "E05"]
unique_failures = set(failures)
print(unique_failures)
print("Number of unique failures:", len(unique_failures))
```

Do not depend on the printed set order.

> **Module III summary:** Data-structure choice is a design decision. Check order, mutability, uniqueness and access method before coding.

MODULE IV

Functional and Modular Programming in Python

Functions reduce repetition; modules and packages organise reusable code.

9 L | 9 P | CO4 | Apply

Module learning objectives

- Use len(), min(), max() and sum().
- Define, call and return values from functions.
- Pass positional and default arguments.
- Explain local/global scope.
- Create/import modules and recognise packages and NumPy.

1. Functions and advantages–

A function is a named reusable block that performs one coherent task. Functions improve decomposition, reuse, readability, testing and maintenance.

Without function	With function
Repeated formulas in multiple places	One tested implementation called many times
Difficult to isolate failures	Each function can be tested independently
Changes require many edits	Change one definition

2. Built-in functions: len(), min(), max(), sum()–

```
readings = [12.4, 12.8, 12.5, 12.9]
print("Count:", len(readings))
print("Minimum:", min(readings))
print("Maximum:", max(readings))
print("Total:", sum(readings))
```

`min()` and `max()` on an empty sequence raise ValueError. Define empty-data behaviour before calculation.

3. User-defined functions, calling and return values–

```
def rectangle_area(length, width):
    area = length * width
    return area

result = rectangle_area(5, 3)
print("Area =", result)
```

`def` creates the function definition. The body runs only when called. `return` ends the function and sends a value to the caller.

Malayalam support: Function-□□□□ result □□□□□□□□□□ □□□□□□□□□□□□□□□□□ `return` □□□□□□□. Function □□□□□□ □□□□□□ `print` □□□□□□□ reuse/test □□□□□□□□□□□ □□□□□□□□□□□□□□□□.

4. Positional and default arguments–

```
def simple_interest(principal, rate=8.0, time=1.0):
    return principal * rate * time / 100

print(simple_interest(10000))
print(simple_interest(10000, 9.5, 2))
```

Arguments are matched by position in these calls. Required parameters must precede parameters with defaults in the definition.

5. Local and global scope–

A name created inside a function is local unless declared otherwise. A module-level name is global. Prefer explicit parameters and return values to hidden global mutation.

```
tax_rate = 0.18            # global name

def total_with_tax(amount):
    total = amount * (1 + tax_rate)  # local total
    return total
```

> Excessive global variables make testing and debugging difficult because a function's result depends on hidden shared state.

6. Module creation and usage–

A module is a Python file. Keep reusable functions in one file and import them into another.

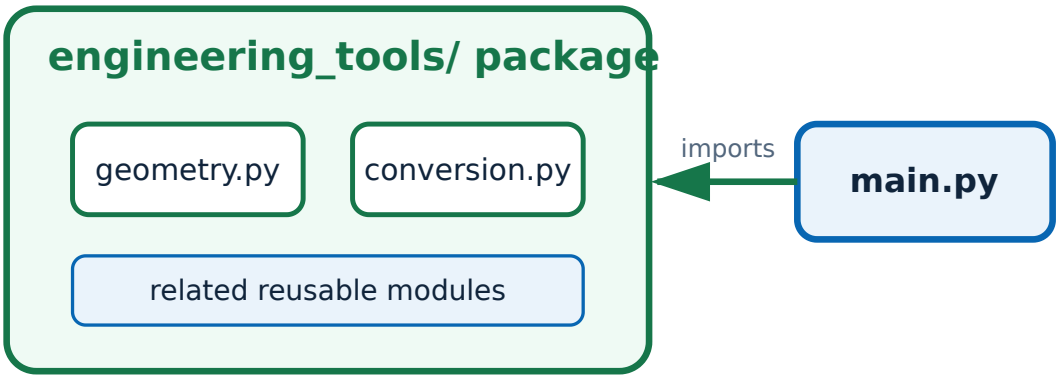
```
# geometry.py
def rectangle_area(length, width):
    return length * width

# main.py
from geometry import rectangle_area
print(rectangle_area(5, 3))
```

> Avoid naming your own file `math.py`, `random.py` or another standard-module name because it can shadow the intended module.

7. Packages and importing packages–

A package groups related modules under a common namespace. Imports can use forms such as `import package.module` or `from package.module import name`. Use explicit imports so the source of each name remains clear.



A package organises related modules; the application imports only what it needs.

8. Introduction to NumPy–

NumPy is a third-party package for numerical arrays and vectorised operations. It is not part of the Python standard library and must be installed into the same interpreter environment used by the IDE.

```
import numpy as np
values = np.array([10, 20, 30])
print(values * 2)
print(values.mean())
```

Python list expression	NumPy array expression	Result
[10,20,30] * 2	np.array([10,20,30]) * 2	List repeats; array performs element-wise multiplication

> Do not claim NumPy is installed merely because code is written. Verify the active interpreter and follow institution-approved installation procedures.

Demonstration: average function

```
def average(values):
    if not values:
        return None
    return sum(values) / len(values)

result = average([72, 84, 90])
print(result)
```

Result = 82.0. Empty input returns None by the stated design.

Function test table

Input	Expected	Reason
[72,84,90]	82	Normal case
[0]	0	Boundary value
[]	None	Empty-data rule

Module IV summary: Give each function one clear responsibility, pass data explicitly, return reusable results and organise tested definitions into modules/packages.

RULE AND PROCEDURE BANK

Python One-Look Rules

Input rule

`input()` returns `str`. Validate and convert.

Range rule

Stop is excluded; step cannot be zero.

Equality rule

== compares values; `is` compares identity.

Mutation rule

List/dict/set are mutable; str/tuple are immutable.

Selection rule

Order mutually exclusive conditions from specific to general.

Loop rule

Verify initial state, condition, update and termination.

Function rule

Clear parameters, one purpose, useful return value.

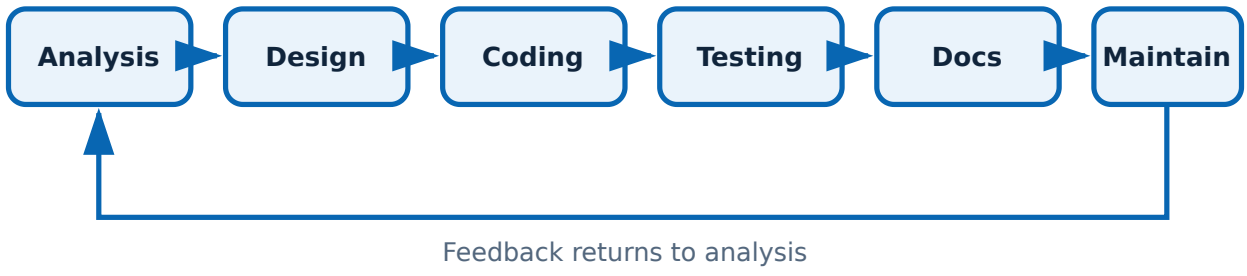
Debug rule

Read the last traceback line, reproduce and isolate.

DIAGRAM AND VISUAL LIBRARY

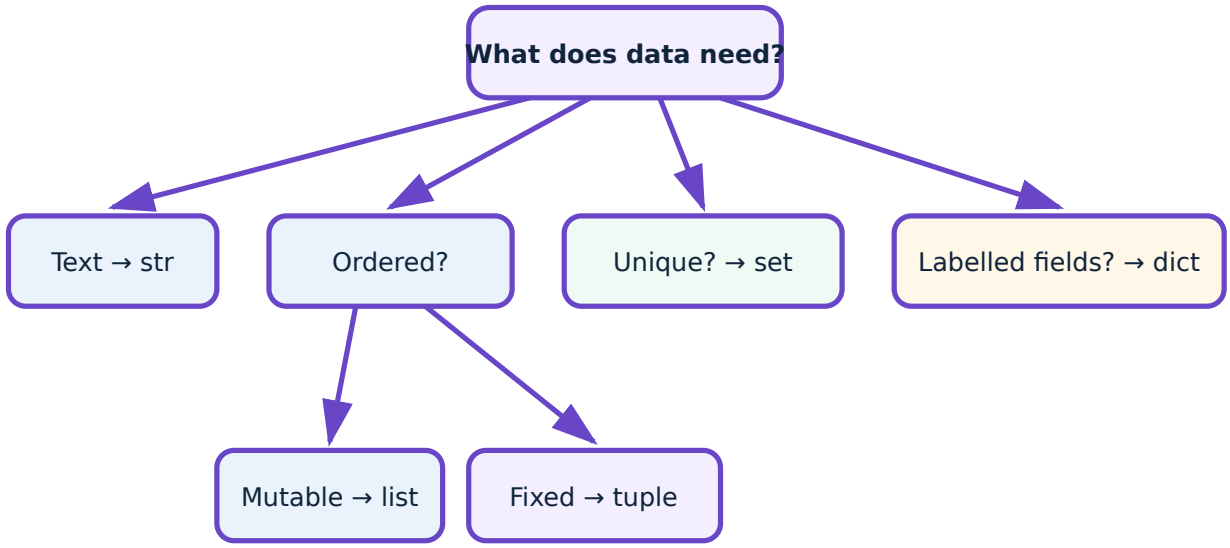
Quick Visual Revision

Program development cycle



Testing and maintenance create feedback; development is not a one-way event.

Data-structure choice



Choose by meaning, not by habit.

PRACTICAL / ACTIVITY / EXPERIMENT CENTRE

Laboratory Experiments and Record-Format Content

The cards below cover every laboratory activity listed in the supplied official major-topics table. Exact software versions and institution-specific field limits must be supplied by the instructor because they are not specified in the PDF.

Experiment 1 Computer laboratory rules, equipment usage and Python environment verification

CO1 | Practicum | Record required

Aim

To identify laboratory rules, use the assigned workstation correctly and verify that Python and an IDE are available.

Tools / software

Assigned computer, institution-approved Python 3 installation, IDLE/PyCharm/VS Code or another approved IDE.

Underlying theory

A safe and reproducible programming environment is the first requirement for practical work. The interpreter executes Python statements; the IDE supports editing, running and debugging.

Safety and laboratory discipline: Use only institution-authorised software and accounts. Do not alter mains wiring, BIOS settings, security controls or network configuration. Save work only in the assigned location.

Procedure

1. Inspect the workstation, power cable and peripherals without opening equipment.
2. Log in with authorised credentials and create the assigned course folder.
3. Open a terminal and run python --version or py --version.
4. Open the approved IDE, create hello.py and enter a one-line print statement.
5. Run the file, record the interpreter version and copy the actual output into the record.
6. Close applications correctly and sign out.

Reference implementation

```
print("Python laboratory ready")
```

Observation / test table

Item	Observation	Status
Python version	_____	Pass / Fail
IDE used	_____	Pass / Fail
Program output	_____	Pass / Fail

Result

The Python environment was verified and a basic program executed successfully.

Precautions

Do not install unapproved extensions. Confirm that the IDE is using the same interpreter verified in the terminal.

Possible errors and troubleshooting: Command not found: Python may not be installed or PATH may be wrong. IDE runs a different version: select the correct interpreter. Permission error: use the assigned folder.

Viva questions

1. **What is the role of the interpreter?**
   It reads and executes Python code and reports errors.
2. **Is an IDE the same as Python?**
   No. The IDE is a development environment; Python is the language implementation/interpreter.

Experiment 2 Algorithm and flowchart for simple interest

CO1 | Practicum | Record required

Aim

To analyse the simple-interest problem and prepare a correct algorithm and flowchart before coding.

Tools / software

Record book, drawing tools or approved flowchart software.

Underlying theory

For principal P, annual rate R percent and time T years, simple interest SI = P × R × T / 100. The algorithm must define input, processing and output.

> **Safety and laboratory discipline:** Use only institution-authorised software and accounts. Do not alter mains wiring, BIOS settings, security controls or network configuration. Save work only in the assigned location.

Procedure

1. State the problem and assumptions.
2. Identify P, R and T as input and SI as output.
3. Write finite numbered algorithm steps.
4. Draw Start, Input, Process, Output and End symbols using arrows.
5. Dry-run with P=10000, R=8 and T=2.
6. Verify that the result is 1600.

Observation / test table

P	R	T	Expected SI	Dry-run SI
10000	8	2	1600	_____
5000	7.5	1	375	_____

Result

A finite algorithm and standards-based flowchart were prepared and verified with test data.

Precautions

Use a parallelogram for input/output and a rectangle for calculation. Include units in the written problem statement.

Possible errors and troubleshooting: Wrong formula order usually comes from omitting /100. Unlabelled arrows or a missing End symbol make the flowchart ambiguous.

Viva questions

1. **Why is dry-running useful?**
   It verifies the logic manually before coding.
2. **Which symbol contains SI = P*R*T/100?**
   A process rectangle.

Experiment 3 Algorithm and flowchart for odd or even number

CO1 | Practicum | Record required

Aim

To design a decision algorithm that classifies an integer as odd or even.

Tools / software

Record book and approved flowchart tool.

Underlying theory

An integer is even when the remainder n % 2 equals zero; otherwise it is odd. A decision diamond must have labelled Yes and No exits.

Safety and laboratory discipline: Use only institution-authorised software and accounts. Do not alter mains wiring, BIOS settings, security controls or network configuration. Save work only in the assigned location.

Procedure

1. Define input n and outputs Even/Odd.
2. Write the condition n % 2 == 0.
3. Prepare steps: Start, input n, test condition, display result, End.
4. Draw and label both decision exits.
5. Dry-run with n=8, n=7 and n=0.

Observation / test table

n	n % 2	Expected class	Observed
8	0	Even	_____
7	1	Odd	_____
0	0	Even	_____

Result

The decision algorithm correctly classified the test integers.

Precautions

Use integer input. Label decision exits; do not draw two independent unlabelled arrows.

Possible errors and troubleshooting: Using / instead of % tests division, not remainder. Missing conversion from input text causes a type error in code.

Viva questions

1. **Is zero even?**
   Yes, because 0 % 2 is 0.
2. **What is modulus?**
   The remainder operator represented by %.

Experiment 4 Basic Python programs: arithmetic, simple interest and temperature conversion

CO1 | Practicum | Record required

Aim

To accept user input, perform arithmetic and display formatted results.

Tools / software

Python 3 and approved IDE.

Underlying theory

input() returns str. Numeric data must be converted. Celsius to Fahrenheit uses F = (9/5)C + 32.

> **Safety and laboratory discipline:** Use only institution-authorised software and accounts. Do not alter mains wiring, BIOS settings, security controls or network configuration. Save work only in the assigned location.

Procedure

1. Create separate files for arithmetic, simple interest and temperature conversion.
2. Prompt clearly for each value and convert using int() or float().
3. Perform calculations with parentheses.
4. Display labels, units and controlled decimal places using f-strings.
5. Test normal, zero and decimal inputs.
6. Record actual output and corrections made.

Reference implementation

```
# Celsius to Fahrenheit
c = float(input("Temperature in °C: "))
f = (9 / 5) * c + 32
print(f"{c:.1f} °C = {f:.1f} °F")
```

Observation / test table

Program	Input	Expected	Actual	Pass/Fail
SI	10000,8,2	1600	_____	__
C→F	25	77	_____	__

Result

The programs accepted input, calculated correctly and displayed meaningful output.

Precautions

Never concatenate a number directly with text without conversion or f-string formatting. Include units.

Possible errors and troubleshooting: ValueError means conversion failed. TypeError may mean text was used in arithmetic. Incorrect 45 °F for 25 °C indicates 9/5 was coded wrongly.

Viva questions

1. **Why use float rather than int for temperature?**
   Temperature may contain decimal values.
2. **What does :.1f mean?**
   Format a number with one digit after the decimal point.

Experiment 5 Decision-making: odd/even, leap year and validated simple interest

CO2 | Practicum | Record required

Aim

To implement conditional decisions and reject invalid fields before calculation.

Tools / software

Python 3 and approved IDE.

Underlying theory

Boolean expressions control if/elif/else. Leap-year logic must handle century years. Validation should check that principal, rate and time satisfy the chosen domain before calculation.

Safety and laboratory discipline: Use only institution-authorised software and accounts. Do not alter mains wiring, BIOS settings, security controls or network configuration. Save work only in the assigned location.

Procedure

1. Write the odd/even program and test positive, negative and zero integers.
2. Write the leap-year program using the complete condition.
3. Write simple-interest input validation; reject non-positive principal/time and out-of-policy rate values according to instructor guidance.

4. Use one clear message for each invalid condition.
5. Run boundary tests and record results.

Reference implementation

```
year = int(input("Year: "))
is_leap = year % 400 == 0 or (year % 4 == 0 and year % 100 != 0)
print("Leap year" if is_leap else "Not a leap year")
```

Observation / test table

Case	Input	Expected branch	Actual
Century leap	2000	Leap	_____
Century non-leap	1900	Not leap	_____
Normal leap	2024	Leap	_____

Result

The programs selected the correct branch and invalid inputs were prevented from reaching the calculation.

Precautions

Order validation from fundamental to specific. Do not invent institutional financial limits; use instructor-approved constraints.

Possible errors and troubleshooting: Using year % 4 == 0 alone misclassifies 1900. Separate if blocks may print multiple messages when elif is required.

Viva questions

1. **Why is 1900 not a leap year?**
   It is divisible by 100 but not by 400.
2. **What is validation?**
   Checking whether input satisfies required type, range and rules before processing.

Experiment 6 Selection-based operations: day name and menu-driven arithmetic

CO2 | Practicum | Record required

Aim

To implement multi-way selection using if-elif or match-case.

Tools / software

Python 3.

Underlying theory

A multi-way selection maps one input choice to one operation. A default branch handles unsupported values and division must check for zero divisor.

> **Safety and laboratory discipline:** Use only institution-authorised software and accounts. Do not alter mains wiring, BIOS settings, security controls or network configuration. Save work only in the assigned location.

Procedure

1. Create a day-number program for values 1 to 7.
2. Create a menu showing addition, subtraction, multiplication and division.
3. Read the choice and operands.
4. Use match-case or if-elif to execute one operation.
5. Add default handling and division-by-zero protection.
6. Test every menu item and one invalid choice.

Reference implementation

```
a = float(input("First number: "))
b = float(input("Second number: "))
choice = input("+, -, *, /: ")
match choice:
    case "+": print(a + b)
    case "-": print(a - b)
    case "*": print(a * b)
    case "/" if b != 0: print(a / b)
    case "/": print("Division by zero is not allowed")
    case _: print("Invalid choice")
```

Observation / test table

Choice	a	b	Expected	Actual
+	8	2	10	____
/	8	0	Error message	____
?	8	2	Invalid choice	____

Result

The program performed exactly one selected operation and handled invalid choices safely.

Precautions

Display the menu before input. Use a default branch. Check zero before division.

Possible errors and troubleshooting: No output often means the choice type/value does not match the cases. Multiple outputs indicate independent if blocks.

Viva questions

1. **What does case _ mean?**
   The default pattern matching any remaining value.
2. **Why guard division?**
   Division by zero raises ZeroDivisionError.

Experiment 7 Looping: multiplication table and pattern printing

CO2 | Practicum | Record required

Aim

To use for loops, range() and nested loops for repetitive tasks.

Tools / software

Python 3.

Underlying theory

A for loop iterates over a sequence. range(start, stop, step) excludes stop. Nested loops use one loop inside another; typically the outer loop controls rows and the inner loop controls items in a row.

Safety and laboratory discipline: Use only institution-authorised software and accounts. Do not alter mains wiring, BIOS settings, security controls or network configuration. Save work only in the assigned location.

Procedure

1. Read an integer and print its multiplication table from 1 to 10.
2. Create a right-triangle star pattern using nested loops.
3. Predict iteration counts before running.
4. Test with positive, zero and negative numbers where meaningful.
5. Record output and explain the outer/inner loop roles.

Reference implementation

```
n = int(input("Number: "))
for i in range(1, 11):
    print(f"{n} × {i} = {n*i}")

rows = 5
for r in range(1, rows + 1):
    for _ in range(r):
        print("*", end=" ")
    print()
```

Observation / test table

Task	Input	Expected iterations	Actual
Table	n=7	10	___
Triangle	rows=5	15 stars	___

Result

The for and nested loops produced the required repetitive output.

Precautions

Remember that stop is excluded. Use end= only when output must stay on the same line.

Possible errors and troubleshooting: One missing line often means an off-by-one range. A pattern on one line means the newline print() is misplaced.

Viva questions

1. **What does range(1,11) generate?**
   Integers 1 through 10.

2. **Which loop controls rows?**
   The outer loop.

Experiment 8 While loop and loop-control statements

CO2 | Practicum | Record required

Aim

To implement condition-controlled iteration and demonstrate break, continue and pass.

Tools / software

Python 3.

Underlying theory

while repeats while its condition is true. break exits; continue advances to the next iteration; pass is a no-operation placeholder.

> **Safety and laboratory discipline:** Use only institution-authorised software and accounts. Do not alter mains wiring, BIOS settings, security controls or network configuration. Save work only in the assigned location.

Procedure

1. Write a while loop that counts from 1 to 5.
2. Write a loop that skips multiples of 3 using continue.
3. Write a search loop that exits with break when a target is found.
4. Use pass in a clearly labelled placeholder branch.
5. Confirm that each loop-control variable is updated correctly.

Reference implementation

```
i = 0
while i < 10:
    i += 1
    if i % 3 == 0:
        continue
    if i > 8:
        break
    print(i)
```

Observation / test table

Statement	Expected effect	Observed
continue	Skip multiples of 3	_____

Statement	Expected effect	Observed
break	Stop after condition	_____
pass	No operation	_____

Result

Condition-controlled loops and loop-control statements were demonstrated without an infinite loop.

Precautions

Update the control variable before any continue that would otherwise bypass the update.

Possible errors and troubleshooting: Infinite loop: update missing or condition never false. Unexpected skipped output: continue occurs too early.

Viva questions

1. **Does pass skip an iteration?**
   No. It does nothing; execution continues with the next statement.
2. **Which statement exits the loop?**
   break.

Experiment 9 String manipulation

CO3 | Practicum | Record required

Aim

To split, count, change case and reverse strings using operations and methods.

Tools / software

Python 3.

Underlying theory

Strings are immutable Unicode sequences. Indexing accesses one character; slicing selects a range; methods return new strings.

Safety and laboratory discipline: Use only institution-authorised software and accounts. Do not alter mains wiring, BIOS settings, security controls or network configuration. Save work only in the assigned location.

Procedure

1. Read a sentence.
2. Display upper and lower case copies.
3. Split into words and count words/characters.
4. Count a selected character.
5. Reverse using slicing.
6. Test empty input and spaces around input.

Reference implementation

```
text = input("Text: ")
clean = text.strip()
print("Upper:", clean.upper())
print("Lower:", clean.lower())
print("Words:", clean.split())
print("Characters:", len(clean))
print("Reversed:", clean[::-1])
```

Observation / test table

Input	Operation	Expected	Actual
Poly PMNA	split()	[Poly, PMNA]	____
Python	[::-1]	nohtyP	____

Result

The program demonstrated required string methods and slicing.

Precautions

Remember methods do not modify the original string. Assign the returned value when needed.

Possible errors and troubleshooting: IndexError occurs when accessing outside the string. Unexpected empty words may come from a specific separator rather than default split().

Viva questions

1. **Why is strip() useful?**
   It removes leading and trailing whitespace.
2. **What is negative indexing?**
   Accessing from the end, such as -1 for the last character.

Experiment 10 List operations and methods

CO3 | Practicum | Record required

Aim

To insert, append, delete, sort, slice and display list data.

Tools / software

Python 3.

Underlying theory

Lists are mutable ordered collections. Methods such as append, insert, remove, pop and sort change the list.

> **Safety and laboratory discipline:** Use only institution-authorised software and accounts. Do not alter mains wiring, BIOS settings, security controls or network configuration. Save work only in the assigned location.

Procedure

1. Create a list of sample marks.
2. Append and insert new values.
3. Delete by value and by index.
4. Sort the list and display a slice.
5. Test membership and count a repeated value.
6. Record the list after every mutation.

Reference implementation

```
marks = [72, 85, 61, 72]
marks.append(90)
marks.insert(1, 78)
marks.remove(61)
removed = marks.pop()
marks.sort()
print("List:", marks)
print("First three:", marks[:3])
print("Count of 72:", marks.count(72))
```

Observation / test table

Step	List state
Initial	_____
After append	_____
After delete	_____
After sort	_____

Result

The list was modified and sliced using the required operations and methods.

Precautions

Do not assign the result of sort() back to the list. Verify a value exists before remove() in robust programs.

Possible errors and troubleshooting: ValueError from remove means the value is absent; IndexError from pop means the index is invalid.

Viva questions

1. **How is append different from insert?**
   append adds at the end; insert adds at a specified index.
2. **What does slicing return?**
   A new list containing the selected elements.

Experiment 11 Dictionary operations

CO3 | Practicum | Record required

Aim

To create, insert, update, delete and display dictionary elements, keys and values.

Tools / software

Python 3.

Underlying theory

A dictionary stores key-value pairs. Keys are unique and normally chosen to represent field names or identifiers.

> **Safety and laboratory discipline:** Use only institution-authorised software and accounts. Do not alter mains wiring, BIOS settings, security controls or network configuration. Save work only in the assigned location.

Procedure

1. Create a student dictionary with name and mark.
2. Insert a new key-value pair.
3. Update an existing value.
4. Display the full dictionary, keys and values.
5. Delete one item with pop() or del.
6. Use get() for a possibly missing key and record the result.

Reference implementation

```
student = {"name": "Anu", "mark": 84}
student["grade"] = "B"
student["mark"] = 88
print(student)
print(list(student.keys()))
print(list(student.values()))
removed = student.pop("grade")
print("Removed:", removed)
print("Phone:", student.get("phone", "Not available"))
```

Observation / test table

Operation	Expected keys	Actual keys
Initial	name, mark	_____
Insert grade	name, mark, grade	_____
Delete grade	name, mark	_____

Result

Dictionary creation, insertion, update, deletion and views were demonstrated.

Precautions

Keys must be hashable and should be meaningful. Use get() when absence is acceptable.

Possible errors and troubleshooting: KeyError means a key was accessed directly but is missing. Wrong update may come from misspelling the key and creating a new one.

Viva questions

1. **Are dictionary values required to be unique?**
   No. Keys are unique; values may repeat.
2. **What does keys() return?**
   A dynamic view of the dictionary keys.

Experiment 12 Set creation and set operations

CO3 | Practicum | Record required

Aim

To create sets, display unique elements and perform union, intersection and difference.

Tools / software

Python 3.

Underlying theory

A set is an unordered collection of unique hashable elements. Union combines, intersection finds common values and difference finds values only in the first set.

Safety and laboratory discipline: Use only institution-authorised software and accounts. Do not alter mains wiring, BIOS settings, security controls or network configuration. Save work only in the assigned location.

Procedure

1. Create two sets with some common and duplicate input values.
2. Display each set and note duplicate removal.
3. Compute union, intersection and both directional differences.
4. Test membership for one value.
5. Record results without assuming display order.

Reference implementation

```
a = {1, 2, 2, 3, 4}
b = {3, 4, 5}
print("A:", a)
print("B:", b)
print("Union:", a.union(b))
print("Intersection:", a.intersection(b))
print("A-B:", a.difference(b))
print("B-A:", b.difference(a))
```

Observation / test table

Operation	Expected mathematical set	Actual
A ∪ B	{1,2,3,4,5}	_____
A ∩ B	{3,4}	_____
A − B	{1,2}	_____

Result

Set uniqueness and the required set operations were demonstrated.

Precautions

Do not compare printed order. Compare membership or set equality.

Possible errors and troubleshooting: Unexpected order is normal. TypeError may occur if an unhashable object such as a list is inserted.

Viva questions

1. **Is A-B the same as B-A?**
   No; set difference is directional.

2. **How is an empty set written?**
   set(), because {} creates an empty dictionary.

Experiment 13 Built-in and user-defined functions with arguments

CO4 | Practicum | Record required

Aim

To apply len(), min(), max(), sum() and create reusable functions using positional and default arguments.

Tools / software

Python 3.

Underlying theory

A function receives arguments through parameters and may return a result. Default parameters provide fallback values.

> **Safety and laboratory discipline:** Use only institution-authorised software and accounts. Do not alter mains wiring, BIOS settings, security controls or network configuration. Save work only in the assigned location.

Procedure

1. Use len, min, max and sum on a numeric list.
2. Define a function that returns average and handles an empty list.
3. Define a simple-interest function with default rate/time.
4. Call the function using different positional argument combinations.
5. Test and record return values.

Reference implementation

```
def average(values):
    if len(values) == 0:
        return None
    return sum(values) / len(values)

def simple_interest(p, r=8.0, t=1.0):
    return p * r * t / 100

print(average([72, 84, 90]))
print(simple_interest(10000))
print(simple_interest(10000, 9.5, 2))
```

Observation / test table

Call	Expected	Actual
average([72,84,90])	82	___
simple_interest(10000)	800	___

Result

Built-in functions and user-defined functions with argument passing were implemented.

Precautions

Place required parameters before default parameters. Return values instead of printing inside reusable calculation functions.

Possible errors and troubleshooting: TypeError may mean a required argument is missing or too many were supplied. None may indicate a missing return statement.

Viva questions

1. **What is a return value?**
   The value sent back to the caller by return.
2. **Can a function have no parameters?**
   Yes, though parameters often improve reuse.

Experiment 14 Custom module creation and use

CO4 | Practicum | Record required

Aim

To create a Python module and import its functions into another program.

Tools / software

Python 3, approved IDE, one working folder.

Underlying theory

A module is a .py file. import executes/loads it and makes its definitions available. The importing file and module must be in an importable location.

Safety and laboratory discipline: Use only institution-authorised software and accounts. Do not alter mains wiring, BIOS settings, security controls or network configuration. Save work only in the assigned location.

Procedure

1. Create calculations.py with two functions.
2. Create main.py in the same folder.
3. Import the functions using import calculations or from calculations import ...
4. Call each function with test data.
5. Run main.py, record output and inspect __pycache__ only if permitted.
6. Rename any file that conflicts with a standard module name.

Reference implementation

```
# calculations.py
def rectangle_area(length, width):
    return length * width

def c_to_f(celsius):
    return (9 / 5) * celsius + 32

# main.py
from calculations import rectangle_area, c_to_f
print(rectangle_area(5, 3))
print(c_to_f(25))
```

Observation / test table

Function	Input	Expected	Actual
rectangle_area	5,3	15	____
c_to_f	25	77	____

Result

The custom module was imported and both functions returned the expected values.

Precautions

Keep both files in the assigned project folder. Avoid filenames such as math.py that shadow standard modules.

Possible errors and troubleshooting: ModuleNotFoundError: file is not in the import path or name differs. AttributeError: function name is misspelled. Circular import: modules import each other unnecessarily.

Viva questions

1. **What is a module?**
   A Python file containing definitions and executable statements.
2. **Why use modules?**
   To organise and reuse tested code without copying it.

Standard record-page format

1. Experiment number and title
2. Aim and mapped CO
3. Tools/software and theory
4. Algorithm and flowchart where applicable
5. Source code with meaningful names/comments
6. Test/observation table
7. Actual output
8. Result and inference
9. Errors found and corrections
10. Viva questions

Never fabricate output. Execute the final code and record the actual result.

EXPECTED / PRACTICE QUESTIONS

Module-wise Question Bank with Model Answers

These are practice questions written from the official topics. They are not claimed as official SITTTR questions.

Module I — Expected / Practice Questions–

Very Short Answer Q1

Define an algorithm.

Model answer: A finite, ordered and unambiguous set of effective steps that converts valid input into the required output.

Short Answer Q2

Differentiate algorithmic and heuristic problem-solving methods.

Model answer: An algorithmic method follows defined steps and is reproducible; a heuristic is a practical rule or approximation that may be quicker but does not guarantee the best or exact result.

Descriptive Q3

Explain the program development cycle.

Model answer: Problem definition → analysis of inputs, outputs and constraints → algorithm/flowchart design → coding → testing and debugging → documentation → maintenance. Each phase produces evidence for the next phase.

Diagram Q4

Name the standard symbols used for Start/End, Input/Output, Process and Decision.

Model answer: Oval/terminator, parallelogram, rectangle and diamond respectively. Arrows show direction of control flow.

Application Q5

Why must test data be designed before coding?

Model answer: It exposes missing requirements and provides normal, boundary and invalid-input cases against which the implementation can be verified.

Short Answer Q6

What does input() return in Python?

Model answer: A string. Convert it with int(), float() or another suitable constructor before numeric operations.

Comparison Q7

Distinguish == and is.

Model answer: == compares values; is compares object identity. Use x is None for a None identity test.

Program Q8

Write the expression for simple interest.

Model answer: interest = principal * rate * time / 100. Principal is the amount, rate is percent per year and time is in years when that convention is used.

Module II — Expected / Practice Questions–

Very Short Answer Q1

What is a selection control structure?

Model answer: A structure that chooses one execution path based on a Boolean condition or pattern.

Short Answer Q2

When is if-elif-else preferable to separate if statements?

Model answer: When alternatives are mutually exclusive and only the first true branch must execute.

Program Logic Q3

State the correct leap-year condition.

Model answer: year % 400 == 0 or (year % 4 == 0 and year % 100 != 0).

Comparison Q4

Compare for and while loops.

Model answer: for is normally used for sequence/range-controlled iteration; while repeats while a condition remains true and requires explicit state update.

Very Short Answer Q5

What values are generated by range(2, 10, 3)?

Model answer: 2, 5 and 8. The stop value 10 is excluded.

Short Answer Q6

Explain break, continue and pass.

Model answer: break terminates the nearest loop; continue skips the rest of the current iteration; pass performs no action and is used as a placeholder.

Debugging Q7

A while loop never ends. What should be checked?

Model answer: Check that the condition can become false, the loop-control variable is updated on every relevant path, and continue does not bypass the update.

Application Q8

Why should input validation occur before interest calculation?

Model answer: It prevents meaningless or unsafe values from reaching the formula and allows a clear error message instead of silently producing invalid output.

Module III — Expected / Practice Questions–

Very Short Answer Q1

What does s[::-1] do for a string s?

Model answer: It returns a reversed copy using a slice with step -1.

Short Answer Q2

Why is a string called immutable?

Model answer: Its characters cannot be changed in place; operations and methods create a new string.

Comparison Q3

Differentiate list and tuple.

Model answer: Both are ordered sequences. A list is mutable and uses square brackets; a tuple is immutable and commonly uses parentheses.

Short Answer Q4

What is the difference between remove() and pop() on a list?

Model answer: remove(value) deletes the first matching value; pop(index) deletes and returns the item at an index, defaulting to the last item.

Application Q5

Choose a structure for unique student roll numbers.

Model answer: A set, because it automatically prevents duplicates and supports membership testing.

Application Q6

Choose a structure for a student record with labelled fields.

Model answer: A dictionary, for example {"name":"Anu", "mark":84}, because values are accessed by meaningful keys.

Program Logic Q7

How can the common elements of sets a and b be obtained?

Model answer: a.intersection(b) or a & b.

Debugging Q8

Why does marks = marks.sort() make marks become None?

Model answer: list.sort() modifies the list in place and returns None. Call

marks.sort() without assignment or use marks = sorted(marks).

Module IV — Expected / Practice Questions–

Very Short Answer Q1

Name the official built-in functions listed in the syllabus.

Model answer: len(), min(), max() and sum().

Short Answer Q2

State two advantages of functions.

Model answer: They reduce repetition, improve readability, simplify testing and support reuse. Any two with explanation are acceptable.

Comparison Q3

Differentiate a parameter and an argument.

Model answer: A parameter is a name in the function definition; an argument is the actual value supplied during a call.

Short Answer Q4

What is a default argument?

Model answer: A parameter with a predefined value used when the caller omits that argument.

Short Answer Q5

Explain local and global scope.

Model answer: A local name is created inside a function and is normally accessible only there; a global name is defined at module level and can be read throughout the module.

Comparison Q6

Differentiate module and package.

Model answer: A module is one Python file; a package organises related modules under one importable namespace.

Application Q7

Why is returning a value usually better than printing inside a calculation function?

Model answer: The result can be tested, reused, formatted differently or combined with other calculations.

Very Short Answer

Q8

What is NumPy?

Model answer: A third-party Python package for efficient numerical arrays and vectorised operations; it is not part of the standard library.

ASSESSMENT PRACTICE

Practice Model Paper — not an official SITTTR model question paper

The supplied PDF provides assessment totals but not an official question-paper pattern or marks distribution. This paper is therefore organised by learning task rather than marks.

1. Define algorithm and explain any four characteristics.
2. Draw a flowchart to classify a number as positive, negative or zero.
3. Write a Python program for simple interest with input validation.
4. Develop a menu-driven arithmetic program using match-case.
5. Use a for loop to print a multiplication table and a nested loop to print a pattern.
6. Write a program demonstrating break, continue and pass.
7. Read a string and display split words, count, upper/lower case and reverse.
8. Demonstrate list insert, append, delete, sort and slice.
9. Create sets and display union, intersection and difference.
10. Create a dictionary record, update and delete fields, then display keys and values.
11. Define a function with positional and default arguments and explain scope.
12. Create and import a custom module; state the difference between a module and package.

Answer-outline checklist–

- Algorithms must be finite, definite and show input/output.
- Flowcharts need correct shapes, labelled decision exits and arrows.
- Programs must convert input, validate where required and label output.
- Loop answers must show correct bounds/update and avoid infinite execution.
- Data-structure answers must demonstrate the named methods/operations.
- Module answers must show two separate files or clearly separated file contents.

QUICK REVISION

One-Look Exam and Practical Revision

Before coding

Input, output, constraints, algorithm, flowchart and test data.

Before submission

Indentation, conversions, condition order, bounds, units, labels and actual output.

Common failures

Unconverted input, wrong range stop, missing while update, sort() assignment, missing return and wrong interpreter.

Module summary		
Module	**Core idea**	**Must be able to do**
I	Design before coding	Algorithm, flowchart, types, operators and I/O
II	Control execution	Select paths and repeat safely
III	Organise data	Choose and manipulate five built-in structures
IV	Reuse code	Functions, arguments, scope, modules and packages

Glossary

Algorithm
Finite ordered steps for solving a problem.

Heuristic
Practical strategy without guaranteed optimum.

Flowchart
Graphical representation of control flow.

Interpreter
Software that executes Python code.

IDE
Environment for editing, running and debugging.

Variable
Name bound to an object.

Literal
Value written directly in source.

Type conversion
Creating a value of another type.

Selection
Choosing a path from a condition.

Iteration
Repeated execution of a block.

Range
Arithmetic sequence object used for iteration.

Mutable
Can change after creation.

Immutable
Cannot change in place.

Index
Position used to access a sequence item.

Slice
Selected sequence range.

Set
Unordered unique collection.

Dictionary
Key-value mapping.

Module
Python source file.

Function
Reusable named block of code.

Package
Organisation of related modules.

Parameter
Name in a function definition.

NumPy
Third-party numerical-array package.

Argument
Value supplied to a function call.

Debugging
Locating and correcting defects.

Scope
Region where a name can be accessed.

References

Official references: Not specified in the supplied official syllabus PDF.

Supplementary learning support: Python 3 documentation supplied with or corresponding to the installed environment; instructor-approved IDE help; institution laboratory manual. These are supplementary and not presented as official syllabus references.